

How the Shopping Copilot Works

TikTok TechJam 2026, Track 4 · a plain-language tour of our submission · score on the public test: 0.9748 out of a practical ceiling of 0.9922

1 The game

A pretend customer wants one product. The product is hidden in a catalog of 50,000 items of clothing, shoes, and jewelry. The customer will not name it.

We may talk for ten rounds. In each round we can do two things: ask one question, and show a list of up to ten products. The moment the hidden product appears in our list, the game ends. We win more points when it appears in an *early* round, and *high* in the list. If it never appears, we score zero for that game.

The judges run 800 such games against our program. No human is involved. Our program is a single Python class with two methods: one that starts a game, and one that answers each round.

2 The key fact: the customer quotes the answer

The customer is not a person. It is a small program, and its source code is public. Before the game, it opens the hidden product's own catalog page and copies out up to four short lines — the material, a feature, the price. These lines are its *clues*. Everything the customer ever says is built from these clues.

So the game is really this: **the customer reads to us from the answer sheet, a line or two at a time — but only if we ask.** Our whole design follows from that sentence. Collect the clues. Find the one product that matches them all. Do it in as few rounds as possible.

One more rule matters. The customer never repeats a clue. Once a line has been read out, it is gone. Whatever we hear, we must keep.

3 The pipeline

Each customer message passes through five steps. Then our answer goes back, and the loop repeats.

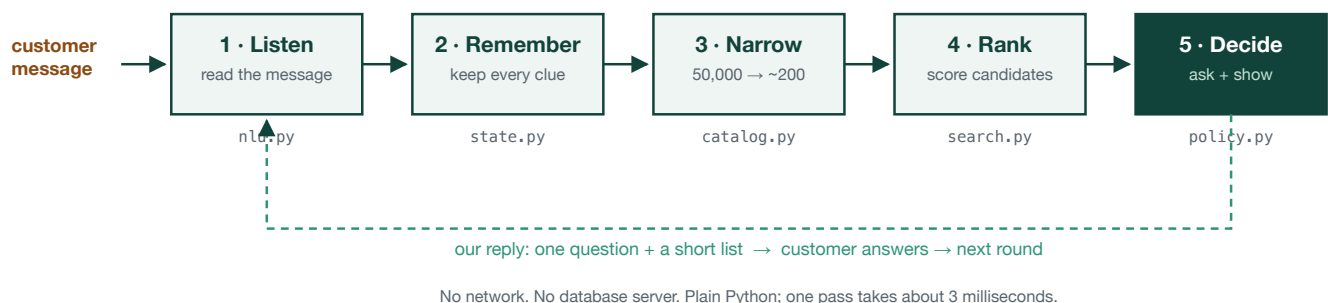


Figure 1. One round of the conversation. The loop runs at most ten times.

4 The five steps, and why each one is the way it is

Step 1 — Listen (three layers)

First we try exact patterns. The public customer program builds its sentences from eight fixed templates, so most messages parse perfectly. If no template fits — the judges may reword messages — a second layer looks for meaning instead: phrases like *“looking for ...”*, *“must have ...”*, *“forget that ...”*. A third layer can ask a language model, but it is switched off by default. The program never needs it, and never needs the internet.

Step 2 — Remember (turn clues down, never throw them away)

Every clue goes into a slot. Sometimes the customer says: *“Actually, ignore my earlier preference.”* The obvious move is to erase the old slots. We measured that, and it is wrong twice over. The customer never repeats a clue, so erased information is gone forever. And the “ignored” preference still describes the same hidden product — the customer changed its wording, not its mind. So we keep the old clues and lower their weight. This one decision repaired the only two games our early prototype lost.

Step 3 — Narrow (the category pool)

The customer’s first sentence names a category: *“I’m looking for Accessories Belts.”* We keep an index from each category to its products. That single sentence cuts 50,000 candidates to a median of 184. In all 200 public games, the hidden product was inside this pool — 200 out of 200.

Step 4 — Rank (evidence, then popularity)

Each candidate in the pool gets a score. Does it contain a clue word for word? All the words of a clue? Some of them? Rare words count more than common ones, so *“pentagram”* outweighs *“necklace”*. Ties are broken by popularity, and popularity is no small thing here: the hidden products are real purchases, and real purchases cluster on popular items. The median hidden product has 6,846 ratings. The median catalog item has 12.

Step 5 — Decide (bet one product at a time)

Two decisions each round: what to ask, and how many products to show.

What to ask. Always the open question — *“anything else that matters?”* The customer holds four clues and hands over up to two per question. Two questions therefore harvest everything. A narrow question (say, only about color) can miss; the open one cannot.

How many to show. One. This is the surprise, so here is the arithmetic. Showing ten products is a bet on all ten at once: if the hidden product is seventh on the list, we are paid as if we guessed seventh — a poor payout. Showing one product per round is a series of single bets: whenever we win, we win at first place. First place instead of seventh is worth 0.26 points; one extra round costs 0.02. The odds are about fifteen to one in favor of patience. We show our single best remaining guess each round, never repeat a guess, and only on the last two rounds do we spread out to full lists, as a safety net.

Switching from “always show ten” to “bet one at a time” moved the score from 0.901 to 0.975 — and raised the average winning position from 0.71 to 0.995, where 1.0 means *every* win at first place.

5 Three real games

These are actual transcripts, replayed through the official judge program.

Game 1 — a buying customer (hidden product: a pentagram pendant necklace)

CUSTOMER	I'm looking for Jewelry Necklaces. A key requirement is: Material:alloy.
COPILLOT	Shows its best single guess (wrong — thousands of necklaces are alloy). Asks: anything else that matters?
CUSTOMER	For that, what matters is: Triple Moon Pentagram Symbol; ...
COPILLOT	Shows the pendant. Hit — round 2, position 1. The rare words did the work.

The first clue was almost useless; the second was a fingerprint. This is why asking beats guessing.

Game 2 — a customer who changes their mind (hidden product: a leather belt)

CUSTOMER	I'm looking for Accessories Belts. Buckle closure
COPILLOT	Best guess shown; asks the open question.
CUSTOMER	For that, what matters is: leather; 100% Leather.
COPILLOT	Shows the right belt... <i>but a hidden rule says a hit before the customer's change of mind does not count. The game continues.</i>
CUSTOMER	Actually, ignore my earlier preference. What I need is: leather.
COPILLOT	Keeps every clue (weights lowered), clears its guess history, and re-ranks.
CUSTOMER	For that, what matters is: Imported; Buckle closure.
COPILLOT	Shows the same belt again. Hit — round 4, position 1. This time it counts.

Had we erased the old clues at the change of mind, only the word *leather* would have remained — and thousands of belts are leather. Keeping the clues is what wins this game.

Game 3 — a browsing customer (hidden product: basketball shorts)

CUSTOMER	I'm looking for Basketball Men, but I'm still exploring.
COPILLOT	Shows the most popular item in that category. Hit — round 1, position 1.

No clues at all — the category plus the popularity prior was enough. This happens more often than you would guess: it is where the 6,846-versus-12 ratio pays off.

6 The score

The official score has three parts: did we find it (weight 0.50), how high on the list (0.30), and how quickly (0.20). Perfect play would score 0.9922, not 1.0 — some rounds are forced by the rules.

Measurement (official judge program, unmodified)	Score
All 200 public games	0.9748
50 held-out games, never used for tuning	0.9648
Every message reworded (robustness test)	0.9566
Half of all clue words replaced by unknown words	0.9248
Official baseline program, for comparison	0.1067

Found: 200 of 200. Winning position: first place in 198 of 200 games. Average winning round: 2.19.

7 What could still go wrong

Our program is tuned to a simulator whose code we can read. The judges' private games use the same code but different products, and they may reword the messages. The two robustness rows above measure exactly that, and the fall is small — the design leans on things that survive rewording: categories, rare words, popularity. With 200 test games, scores carry a noise of about ± 0.03 . So our honest prediction for the private test is **about 0.95, plus or minus 0.03** — a range, not a point.

Repository: `agent.py` (entry) · `src/` (the five steps) · `tools/` (judge runner, robustness suites) · `tests/` . No dependencies; Python 3.9+; fully offline. Every number in this document is reproducible with one command listed in the README.